

Topic 3A: The Transformer Architecture

Last updated: September 22, 2026

Surbhi Goel

Acknowledgements: These notes draw on Bengio et al. [1] for neural language models, Bahdanau, Cho, and Bengio [2] and Vaswani et al. [3] for attention and Transformers, and Ba, Kiros, and Hinton [6] and Xiong et al. [7] for normalization.

AI Disclosure: These notes were drafted and revised with AI assistance. The present revision awaits instructor review. If you find an error, please let me know.

Today we will see how a Transformer takes a sequence of tokens and produces a probability distribution for the next token. We focus on a *causal*, or *decoder-only*, Transformer: each position can use only the prefix ending at that position.

1 From Tokens to Probabilities

Recall from the overview lectures that a tokenizer maps text to tokens $x_1, \dots, x_T$ from a fixed vocabulary $\mathcal{V}$. The model assigns next-token probabilities for each prefix $x_{1:i}$.

Token embeddings. We first associate each token w with a learned vector $e_w \in \mathbb{R}^d$, called its *embedding*. The dimension d is the model width. We store these vectors as the rows of an *embedding matrix* $E \in \mathbb{R}^{|\mathcal{V}| \times d}$. In particular, if $u_w \in \mathbb{R}^{|\mathcal{V}|}$ is the one-hot column vector for token w , then

$$e_w = E^\top u_w \in \mathbb{R}^d.$$

This simply selects row w of E , written as a column vector. We use the same matrix at every position and learn its entries along with the rest of the model's parameters.

The embedding of a token is the same regardless of the words before it. If we predicted using only e_{x_i} , we would assign the same next-token probabilities to all prefixes ending in x_i . For example, consider these prefixes:

Lily's ball is red. The color of the ball is
Lily's ball is blue. The color of the ball is

Both prefixes have the same last token, but their desired continuations start with *red* and *blue*, respectively. The last token's embedding alone cannot distinguish these cases.

Using the preceding tokens. We need to compute a vector using the preceding tokens as well as x_i . Write

$$h_i = f_\theta(x_{1:i}) \in \mathbb{R}^d.$$

Here i is the last position the model has read, and we use h_i to predict x_{i+1} . Unlike the token embedding, this *contextual representation* depends on the preceding text: changing that text can change h_i even when x_i stays the same. Its dimension d is fixed; f_θ specifies how we combine information from the prefix.

From a vector to probabilities. Once we have h_i , we multiply by a learned output matrix and add a bias to produce one score for each token in the vocabulary:

$$\ell_i = W_{\text{out}} h_i + b_{\text{out}}, \quad W_{\text{out}} \in \mathbb{R}^{|\mathcal{V}| \times d}, \quad \ell_i \in \mathbb{R}^{|\mathcal{V}|}. \quad (1)$$

These scores are called *logits*. We then apply softmax over the vocabulary to obtain the next-token distribution:

$$q_\theta(w \mid x_{1:i}) = \frac{\exp(\ell_i(w))}{\sum_{v \in \mathcal{V}} \exp(\ell_i(v))}. \quad (2)$$

The output map and softmax form the *readout*. We now need to choose how to compute h_i .

2 From a Fixed Window to Attention

A fixed window. One choice is to concatenate the embeddings of the last $n - 1$ tokens and apply a feed-forward network g_θ . This gives a neural n -gram model [1]:

$$h_i = g_\theta([e_{x_{i-n+2}}; \dots; e_{x_i}]) \in \mathbb{R}^d.$$

At the beginning of a sequence, we fill any missing slots with a fixed padding vector. Concatenation keeps track of order, since each relative position occupies its own coordinates. A small value of n can leave out information needed for the prediction. Using the subword tokenization in Figure 1, a four-token window gives the same input for both prefixes above:

$$[\text{of}; \text{the}; \text{ball}; \text{is}].$$

The color is outside this window, so the two predictions must be identical. A longer window would reach it in this example, but information outside the chosen window still cannot affect the prediction. The input to g_θ has $(n - 1)d$ coordinates, while its output has d . For a fixed hidden width, enlarging the window adds parameters and computation to the first dense layer. Once we choose that layer, its input size is fixed: we cannot supply a longer window to the same network.

Reading from the prefix. For the color prediction, we would like to use the earlier tokens that specify the color, wherever they occur in the prefix. We can read from these positions using a weighted average. A weight near one almost selects a single position, while several nonzero weights combine information from several positions. Write $z_j \in \mathbb{R}^d$ for the vector at position j , initially $z_j = e_{x_j}$, and form

$$o_i = \sum_{j=1}^i \alpha_{ij} z_j, \quad \alpha_{ij} \geq 0, \quad \sum_{j=1}^i \alpha_{ij} = 1.$$

The output o_i has d coordinates regardless of the prefix length. We can feed it into a fixed-size map to compute h_i , even when the prefix grows. Any token can affect the average, but the average need not preserve all the information in the input vectors.

How should we choose the weights? If α_{ij} depends only on the positions i and j , it gives the same weight to a color token and to an unrelated token occupying the same position. The useful token can also move when we change the text. We therefore compute the weights from the input vectors, allowing the model to change which positions it emphasizes. This operation is called *attention* [2]. When we read from the same sequence we are updating, it is *self-attention* [3].

To choose the weights, we first assign a score s_{ij} to position j when updating position i . A simple choice is the dot product $z_i^\top z_j$. These scores can be negative, so dividing by their sum would

not necessarily give averaging weights. Softmax first exponentiates the scores and then normalizes them:

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{r=1}^i \exp(s_{ir})}, \quad 1 \leq j \leq i. \quad (3)$$

Observe that the ratio of two weights depends only on the difference between their scores:

$$\frac{\alpha_{ij}}{\alpha_{ir}} = \exp(s_{ij} - s_{ir}).$$

Equal scores therefore give equal weights, and increasing one score increases its weight relative to the others. Since softmax is differentiable, we can learn the parameters used to compute the scores by gradient descent.

Queries and keys. The dot product $z_i^\top z_j$ is a restrictive choice. It is symmetric, even though the information we need at i may differ from what we need at j , and it gives us no choice about which features to compare. For example, the score for $j = i$ is always the squared norm $\|z_i\|^2$. We can make the comparison more flexible by learning two linear maps:

$$\mathbf{q}_i = W_Q z_i, \quad \mathbf{k}_j = W_K z_j, \quad W_Q, W_K \in \mathbb{R}^{d_{\text{head}} \times d}.$$

We call these the *query* at position i and the *key* at position j . In the color example, we would like the query at the final *is* to match keys from color words. We compare the query and key using their scaled dot product:

$$s_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_{\text{head}}}} = \frac{z_i^\top W_Q^\top W_K z_j}{\sqrt{d_{\text{head}}}}. \quad (4)$$

Here d_{head} is the number of coordinates in each query and key. The learned matrix $W_Q^\top W_K$ need not be symmetric, so s_{ij} and s_{ji} can differ.

Scaling the scores. We have introduced a factor of $1/\sqrt{d_{\text{head}}}$ in the scores. To see why, suppose that at initialization the coordinates of $\mathbf{q}_i$ and $\mathbf{k}_j$ are all independent, with mean zero and variance one. Then

$$\mathbb{E}[\mathbf{q}_i^\top \mathbf{k}_j] = 0, \quad \text{Var}(\mathbf{q}_i^\top \mathbf{k}_j) = \sum_{r=1}^{d_{\text{head}}} \text{Var}(\mathbf{q}_{i,r} \mathbf{k}_{j,r}) = d_{\text{head}}.$$

Under these assumptions, dividing by $\sqrt{d_{\text{head}}}$ makes the score variance one. Without this scaling, increasing the dimension can concentrate the softmax weights and make its derivatives small. This was the motivation for the scaling in the original Transformer [3, Section 3.2.1]. The scale can still change during training.

Figure 1 uses the subword tokenizer from the lecture slides. The larger illustrative scores on `ur` and `ed`, the two tokens of *red*, give those positions most of the weight in the average.

Values and attention heads. The weights tell us where to read. We still have a choice about what information to take from those positions. We could keep averaging the input vectors z_j . A learned *value* map lets us instead select or transform the features to pass on. For example, keys can emphasize features useful for matching a color query, while values retain information needed to predict the color. These are different roles, so we allow different learned maps:

$$\mathbf{q}_i = W_Q z_i, \quad \mathbf{k}_j = W_K z_j, \quad v_j = W_V z_j, \quad W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{head}} \times d}. \quad (5)$$

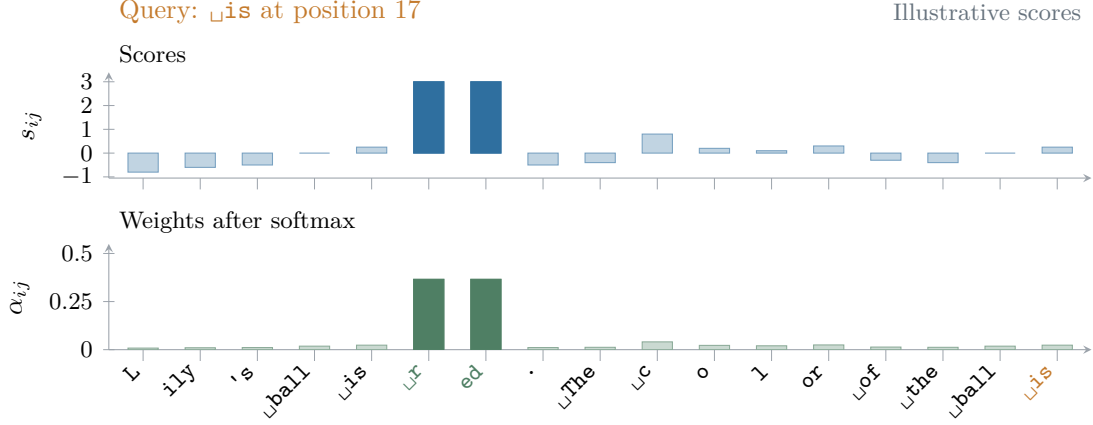


Figure 1: Illustrative attention on subword tokens, with the query at the final `is`. The symbol `l` marks a leading space. The lower panel applies softmax to the scores in the upper panel.

The attention output is now

$$o_i = \sum_{j=1}^i \alpha_{ij} v_j, \quad o_i \in \mathbb{R}^{d_{\text{head}}}. \quad (6)$$

We learn all three maps from the next-token objective. Taking $d_{\text{head}} = d$ and $W_V = I$ recovers the weighted average of the original vectors.

These operations together form an *attention head*. The learned maps W_Q, W_K, W_V are shared across positions; the attention weights are computed for each input.

The causal mask and matrix form. So far we have restricted the sum to $j \leq i$. This is important: when predicting x_{i+1} , we cannot use x_{i+1} itself or any later token. We enforce this restriction with a *causal mask*.

We can compute the attention outputs at all positions together using matrix operations. Stack the input vectors as rows of a matrix $Z \in \mathbb{R}^{T \times d}$, with $z_i^\top$ in row i . The query, key, and value matrices are then

$$Q = ZW_Q^\top, \quad K = ZW_K^\top, \quad V = ZW_V^\top, \quad Q, K, V \in \mathbb{R}^{T \times d_{\text{head}}}.$$

The (i, j) entry of $QK^\top$ is the dot product between query i and key j . To exclude future positions, define a mask matrix $M \in (\mathbb{R} \cup \{-\infty\})^{T \times T}$ by

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

We can now write the whole head computation as

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}} + M\right) V \in \mathbb{R}^{T \times d_{\text{head}}}. \quad (7)$$

Here softmax acts separately on each row, and $\exp(-\infty) = 0$. The i -th row of O is $o_i^\top$ from (6).

Notice that we use softmax in two different places. Here it gives weights over *input positions*; in the readout in (2), it gives probabilities over *vocabulary tokens*. Figure 2 shows how the attention weights are computed and then used to combine the value vectors.

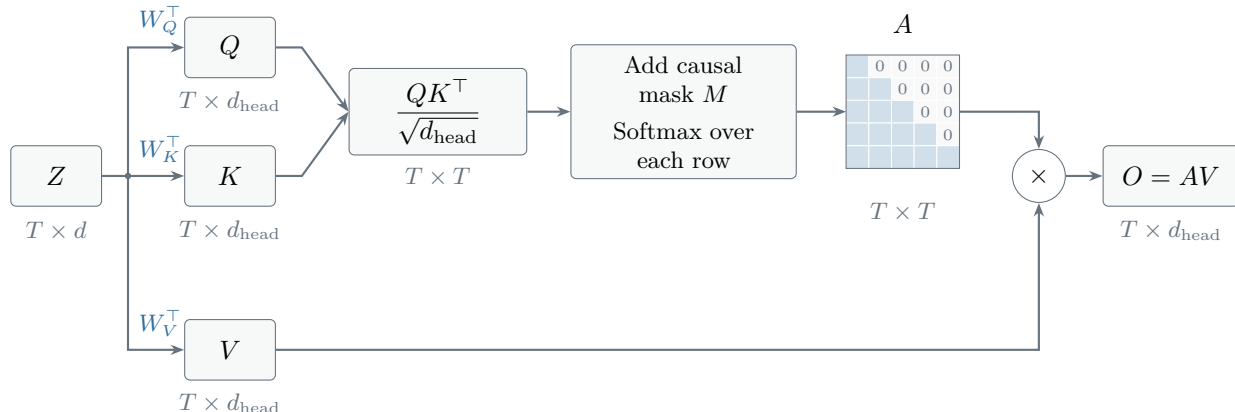


Figure 2: The two computations in an attention head. Queries and keys produce $A = \text{softmax}(QK^\top / \sqrt{d_{\text{head}}} + M)$; multiplying by V then combines value vectors. The triangular grid shows the support of A : blue entries are allowed, and future positions have weight zero. Shading does not indicate weight magnitude.

3 Representing Position

An attention head compares a query with each key and averages the corresponding values. If we hold the query fixed and reorder the key–value pairs, each value moves together with its score. To see why the output does not change, write the weighted average as

$$o_i = \frac{\sum_{j=1}^i \exp(s_{ij}) v_j}{\sum_{j=1}^i \exp(s_{ij})}.$$

Reordering the pairs only reorders the terms in these two sums; their values are unchanged. Thus the output depends on which pairs are available, but not on their order. This is *permutation invariance* with respect to the key–value pairs at a fixed query.

When the inputs are token embeddings alone, rearranging the preceding tokens simply reorders these pairs. This can prevent us from making the right prediction. Consider the two prefixes

- A:** *One day, Lily gave the ball to Tom. Now the ball belongs to*
B: *One day, Tom gave the ball to Lily. Now the ball belongs to*

We should predict Tom in the first case and Lily in the second. Taking each name to be a single token, the two prefixes contain the same tokens. Only Lily and Tom exchange positions, and the last token is the same in both.

With token embeddings as inputs, the final query is the same in both cases and can read all tokens in the prefix. Swapping the names just exchanges their key–value pairs. By the calculation above, the output o_i is unchanged, so a prediction based on this output cannot distinguish the two prefixes.

This argument concerns a single attention layer on token embeddings. In later causal layers, the keys and values can already depend on different prefixes, so reordering the tokens need not simply reorder these pairs.

Position vectors. We can give the model explicit position information by adding a *positional encoding* $p_i \in \mathbb{R}^d$ to each token embedding:

$$z_i^{(0)} = e_{x_i} + p_i. \tag{8}$$

Here the superscript 0 denotes the initial representation, before we apply attention. The query and key projections can now use both the token and its position. When we swap Lily and Tom, we add a different position vector to each name’s embedding.

Learned and sinusoidal positions. We can learn one vector for each supported position, giving a table with $T_{\max}d$ parameters for $T_{\max}$ positions. This does impose a maximum position: going beyond it requires extending the table. The difference from the fixed-window model is how we use these vectors. Concatenation enlarged the input to a dense layer. Here we add p_i to a d -dimensional token vector, so the attention and feed-forward maps keep the same input dimension and the same parameters for every sequence length up to $T_{\max}$.

We can avoid learning a separate vector for each position by using a fixed formula. The original Transformer used sine and cosine pairs at several frequencies [3, Section 3.5]. We can write these as

$$p_i = (\cos(\omega_1 i), \sin(\omega_1 i), \dots, \cos(\omega_{d/2} i), \sin(\omega_{d/2} i)),$$

for even d , with $\omega_m = 10000^{-2(m-1)/d}$ for $m = 1, \dots, d/2$. At position i , we take one value from each curve to form p_i . The frequencies vary on different length scales: some coordinates change quickly from one token to the next, while others vary slowly across a long span. Figure 3 shows three coordinate pairs. An offset r rotates each pair by the fixed angle $\omega_m r$.

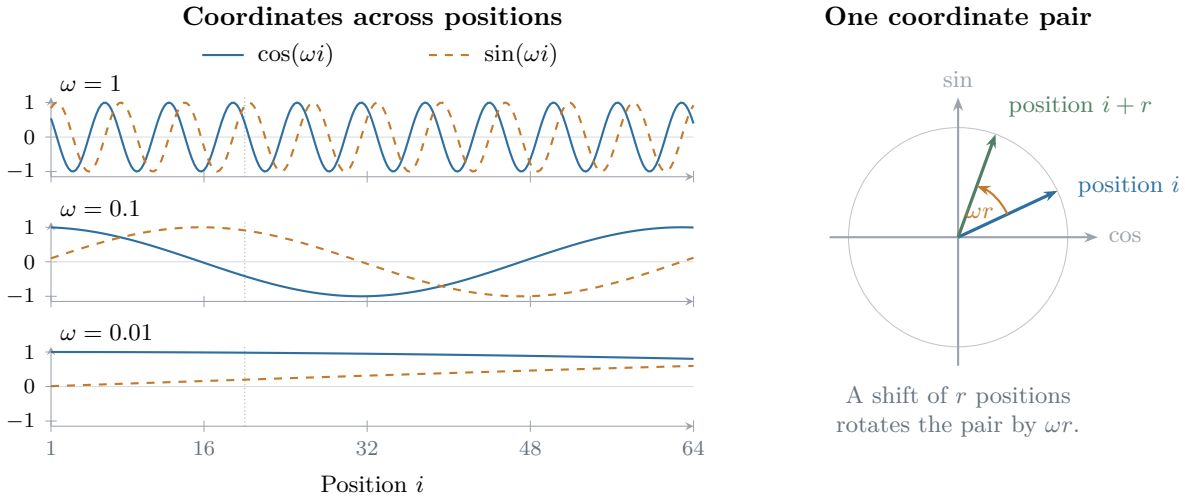


Figure 3: Three sine–cosine pairs from a $d = 64$ encoding. A vertical slice gives six coordinates of p_i . Each pair traces a circle as the position changes; its frequency sets the rotation per token.

Relative positions. Often we care about where a token is relative to the current one. For example, a head that reads the preceding token should use the same positional rule at positions 10 and 100. Both have $j - i = -1$. Absolute position vectors give separate descriptions of i and j ; using their relative offset directly lets us share a positional rule when a passage moves within a document. The content at the two positions can still affect the score.

One way to do this is to add a learned bias b_{j-i} to the score. With rotary position embeddings (RoPE), we instead rotate query and key coordinates according to their positions [4]. For one pair of coordinates, let R_θ be the two-dimensional rotation through angle θ . Rotating the query at i by

ωi and the key at j by ωj gives

$$(R_{\omega i} \mathbf{q}_i)^\top (R_{\omega j} \mathbf{k}_j) = \mathbf{q}_i^\top R_{\omega(j-i)} \mathbf{k}_j,$$

since $R_\theta^\top R_\phi = R_{\phi-\theta}$. The explicit position dependence is through the relative offset $j - i$. RoPE applies this calculation to coordinate pairs at different frequencies.

Using a fixed formula lets us evaluate the encoding at positions beyond those seen in training, though it does not guarantee accurate predictions there. For the rest of these notes, we will use the additive position vectors in (8).

4 The Transformer Block

A Transformer block combines several attention heads with a feed-forward network at each position. We will build these two parts, then add residual connections and layer normalization so that we can stack many blocks.

Multiple attention heads. Recall that all coordinates of a head's output use the same weights α_{ij} . We might want to read different features from different positions, but one head has to use the same weighting for all of them. We can allow several choices of weights by using multiple heads and keeping their outputs in separate coordinates.

Let H be the number of heads and take $d_{\text{head}} = d/H$, assuming H divides d . Each head has its own query, key, and value matrices. At position i , head a produces $o_i^{(a)} \in \mathbb{R}^{d_{\text{head}}}$. We concatenate the outputs and apply an output projection:

$$a_i = W_O \begin{pmatrix} o_i^{(1)} \\ \vdots \\ o_i^{(H)} \end{pmatrix}, \quad W_O \in \mathbb{R}^{d \times d}. \quad (9)$$

The concatenated vector has dimension $H d_{\text{head}} = d$. The projection W_O combines the heads' outputs into an update $a_i \in \mathbb{R}^d$.

We write $\text{MHA}(Z)$ for the matrix whose i -th row is $a_i^\top$. Thus $\text{MHA}(Z)$ has the same shape as Z . Each head uses the causal mask in (7). Figure 4 shows how the head outputs are combined.

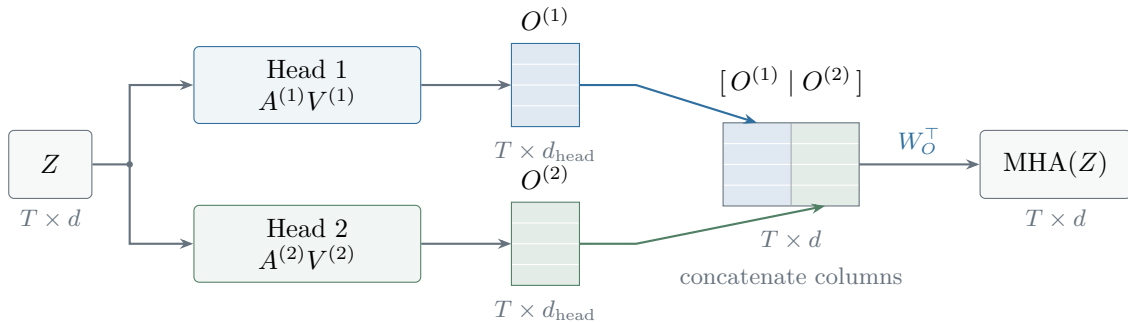


Figure 4: Two heads, each with its own query, key, and value maps. Each computes its own attention matrix $A^{(a)}$ from the same input. Their outputs are concatenated along the feature dimension; $W_O^\top$ then combines the columns. For the two heads shown, $d_{\text{head}} = d/2$. Row i of the result is $a_i^\top$.

We can understand the role of each head by writing the update directly in terms of the input vectors. Partition $W_O = [W_O^{(1)} \dots W_O^{(H)}]$ into column blocks, where $W_O^{(a)} \in \mathbb{R}^{d \times d_{\text{head}}}$. With $B_a = W_O^{(a)} W_V^{(a)} \in \mathbb{R}^{d \times d}$, we have

$$a_i = \sum_{a=1}^H \sum_{j=1}^i \alpha_{ij}^{(a)} B_a z_j. \quad (10)$$

This separates two choices: the query-key product $(W_Q^{(a)})^\top W_K^{(a)}$ determines the attention weights, while B_a determines how features at a source position contribute to the update [9]. Each product has rank at most d_{head} . Although the expression is a weighted sum of linear transformations, the update is generally nonlinear because the weights $\alpha_{ij}^{(a)}$ depend on the input.

The feed-forward network. Attention lets us gather information from other positions. We can now apply a nonlinear function to the features we have gathered at each position. We use a *position-wise feed-forward network* consisting of two affine maps with a coordinatewise nonlinearity ϕ in between:

$$\text{FFN}(z) = W_2 \phi(W_1 z + b_1) + b_2, \quad (11)$$

where

$$W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}, \quad W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}, \quad b_1 \in \mathbb{R}^{d_{\text{ff}}}, \quad b_2 \in \mathbb{R}^d.$$

The intermediate width d_{ff} is often larger than d . The original Transformer used $d_{\text{ff}} = 4d$ and the coordinatewise ReLU activation $\phi(z) = \max\{0, z\}$.

For example, if two coordinates contain bits $b, c \in \{0, 1\}$, the ReLU unit $\max\{0, b + c - 1\}$ computes their conjunction. The matrix W_2 maps the hidden activations back to $\mathbb{R}^d$.

We apply the same feed-forward network independently at every position, so this step does not read from other positions. As with attention, we write $\text{FFN}(Z)$ for the operation on the whole matrix, with the function above applied to each row.

Residual connections. Rather than replacing a representation with the output of a sublayer, we can add the output to the existing representation. This is a *residual connection*:

$$z \mapsto z + F(z).$$

Both multi-head attention and the feed-forward network return vectors in $\mathbb{R}^d$, so we can use this addition after each. In particular, setting F to zero leaves z unchanged. The vectors we carry through these additions form the *residual stream*.

To see how the addition affects gradients, write $y = z + F(z)$ and $g_y = \nabla_y \mathcal{L}$. If DF denotes the Jacobian of F , the chain rule gives

$$g_z = (I + DF(z))^\top g_y = g_y + DF(z)^\top g_y.$$

Without the residual connection, we would have $g_z = DF(z)^\top g_y$, which can become small when $DF(z)$ is small. With the residual connection, we get the extra term g_y .¹ Residual networks were developed by He et al. [5] to make deep networks easier to optimize.

¹This does not guarantee a nonzero gradient; the two terms can cancel.

Layer normalization. As we repeatedly update the representations, their scale can change. This matters for attention: with bias-free query and key maps, multiplying every input vector by $c > 1$ multiplies the scores by c^2 . We can then get much more concentrated softmax weights even though the vector directions have not changed.

We control this scale using *layer normalization* [6]. For each vector, we subtract the mean of its coordinates and divide by their standard deviation, with a small correction to avoid division by zero. We then apply a learned scale and shift. Formally, for $z \in \mathbb{R}^d$, define

$$\mu(z) = \frac{1}{d} \sum_{r=1}^d z_r, \quad \sigma^2(z) = \frac{1}{d} \sum_{r=1}^d (z_r - \mu(z))^2.$$

The normalized vector is

$$\text{LN}(z) = \gamma \odot \frac{z - \mu(z)\mathbf{1}}{\sqrt{\sigma^2(z) + \epsilon}} + \beta, \quad (12)$$

where $\gamma, \beta \in \mathbb{R}^d$ are learned parameters, $\odot$ denotes coordinatewise multiplication, and $\epsilon > 0$ prevents division by zero. The mean and variance are computed separately at each token position, over its d coordinates.

There is also a choice of where to normalize. We will use *pre-normalization*: we normalize the input to each attention or feed-forward sublayer, then add its output to the unnormalized residual representation. The original Transformer instead normalized after each residual addition. These two placements have different effects on gradients and training stability [7].

Putting it all together. We now combine these operations into a block and stack L such blocks. Write $z_i^{(\ell)}$ for the representation at position i after block ℓ , and let row i of $Z^{(\ell)} \in \mathbb{R}^{T \times d}$ be $(z_i^{(\ell)})^\top$. Thus $Z^{(0)}$ contains the token and position embeddings from (8). Block ℓ applies the two updates

$$U^{(\ell)} = Z^{(\ell-1)} + \text{MHA}_\ell(\text{LN}_{\ell, \text{attn}}(Z^{(\ell-1)})), \quad (13)$$

$$Z^{(\ell)} = U^{(\ell)} + \text{FFN}_\ell(\text{LN}_{\ell, \text{ff}}(U^{(\ell)})). \quad (14)$$

The output $Z^{(\ell)}$ becomes the input to the next block. All three matrices have shape $T \times d$. Each block has its own parameters, which we share across positions; the two normalizations also have separate scale and shift parameters. Figure 5 shows the two residual paths.

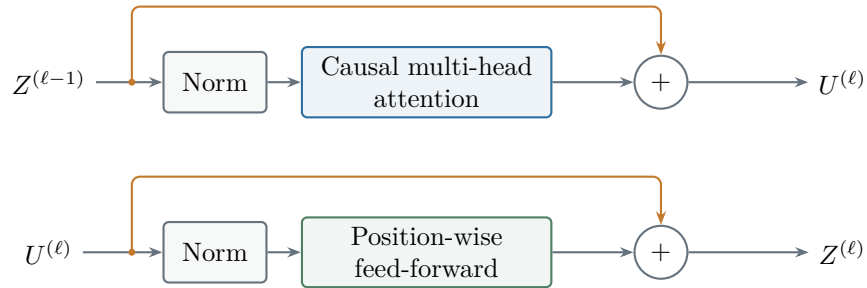


Figure 5: The two successive updates of a pre-normalized Transformer block. The orange paths carry the existing representations directly to the additions. Normalization is applied before each sublayer. Only the attention sublayer communicates between token positions.

What do we gain by stacking blocks? One causal attention layer already lets position i read every $j \leq i$. In the first block, however, we form queries and keys from normalized token and

position embeddings. In later blocks, we form them from representations that already contain information gathered by earlier blocks. We can therefore choose what to read based on what we have already computed [9]. Figure 6 shows this dependence across two blocks. Within a single attention sublayer, all positions read the input vectors to that sublayer, not the outputs being computed at other positions.

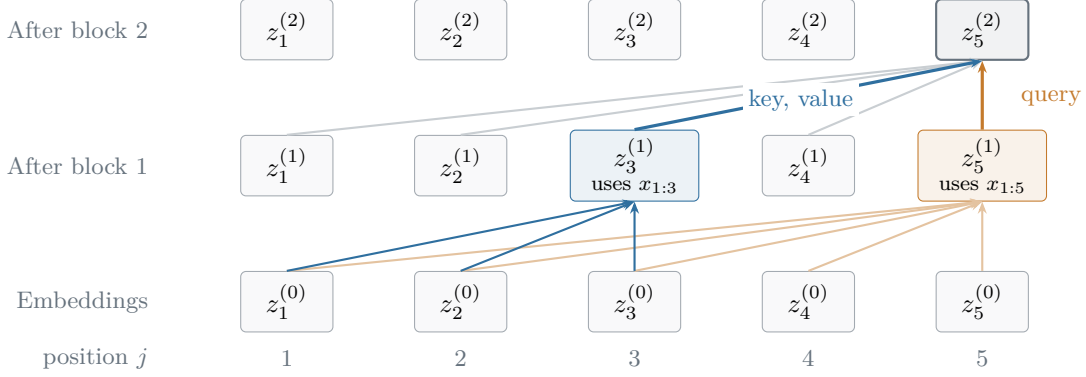


Figure 6: Selected dependencies through two blocks. In block 2, position 5 forms its query from a representation of $x_{1:5}$ and can read a key and value at position 3 that already depend on $x_{1:3}$. One block can access the whole prefix; another makes the attention computation depend on earlier computations. All arrows cross between blocks, and omitted arrows are also allowed when causal.

Causality. We should check that computing all positions together does not let the model use future tokens. We can show this by induction over the blocks. Initially, $z_i^{(0)}$ depends only on x_i and its position. Now suppose the vector at each position j after block $\ell - 1$ depends only on $x_{1:j}$. Causal attention at i reads only positions $j \leq i$, so its output depends only on $x_{1:i}$. Normalization, residual addition, and the feed-forward network act separately at each position and preserve this property. Thus our prediction at i depends only on the prefix $x_{1:i}$, even when we process the whole training document in one forward pass.

The readout. After all L blocks, we apply a final normalization to obtain the prefix representation $h_i = f_\theta(x_{1:i}) = \text{LN}_{\text{final}}(z_i^{(L)})$. We then use the readout from (1)–(2) to predict x_{i+1} :

$$\ell_i = W_{\text{out}} h_i + b_{\text{out}}, \quad q_\theta(\cdot | x_{1:i}) = \text{softmax}(\ell_i).$$

The parameters θ include all the learned matrices, biases, and normalization parameters.

We can reuse the input embedding matrix at the output by setting $W_{\text{out}} = E$. This is called *weight tying*, and gives the following logit for token w :

$$\ell_i(w) = e_w^\top h_i + b_{\text{out}}(w).$$

The same row of E now has two roles: it represents token w at the input and scores w against the final contextual representation at the output. We can also leave the matrices untied and learn the input embeddings and output scoring vectors separately. Figure 7 shows the complete computation for all positions together.

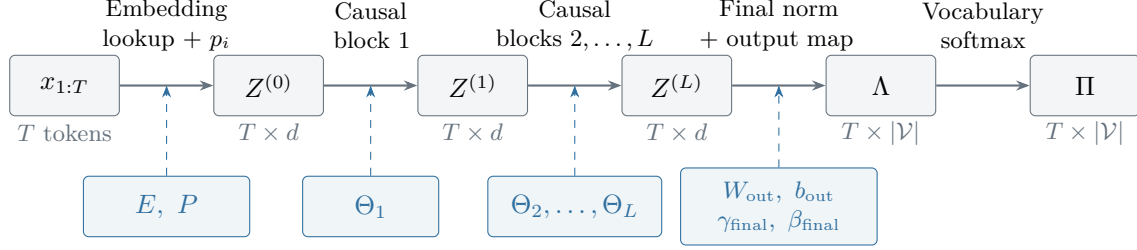


Figure 7: The full forward computation, shown with learned additive position vectors stored in P . Gray boxes show the input and computed values; blue boxes are learned parameters shared across positions. Each block has its own parameters Θ_ℓ . Row i of Λ contains the logits $\ell_i^\top$; row i of Π is $q_\theta(\cdot \mid x_{1:i})$, the distribution for x_{i+1} .

5 Parameters and Computation

Parameter counts. We can now count the parameters in the model. A single head has $3dd_{\text{head}}$ entries in its query, key, and value matrices. With H heads of width $d_{\text{head}} = d/H$, the query projections together have $Hdd_{\text{head}} = d^2$ entries. We get another d^2 each from the key, value, and output projections. The two feed-forward matrices contribute $2dd_{\text{ff}}$ entries.

Table 1 collects these counts and the arithmetic cost for an input of length T . We omit biases and normalization parameters, which contribute $O(d + d_{\text{ff}})$ parameters per block.

Operation	Matrix parameters	Arithmetic cost
Query, key, and value projections	$3d^2$	$O(Td^2)$
Attention scores and weighted averages	0	$O(T^2d)$
Multi-head output projection	d^2	$O(Td^2)$
Feed-forward network	$2dd_{\text{ff}}$	$O(Tdd_{\text{ff}})$

Table 1: Parameter counts and dense arithmetic costs for one block. Attention weights are computed from the input and add no parameters.

With $d_{\text{ff}} = 4d$, one block has $12d^2$ matrix parameters: $4d^2$ in attention and $8d^2$ in the feed-forward network. Thus two thirds of the block’s matrix parameters are in the feed-forward network.

For L such blocks and tied input and output embeddings, the main parameter count is

$$12Ld^2 + |\mathcal{V}|d.$$

Here we count the embedding matrix E only once because the output matrix W_{out} shares its entries. If we instead learn $W_{\text{out}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ separately from E , we add another $|\mathcal{V}|d$ parameters; this is the *untied* case. A learned position table adds $T_{\text{max}}d$, whereas fixed sinusoidal encodings add none.

Parallel computation. During training, we have all the input tokens. Since queries, keys, and values within a sublayer come from the same input matrix, we can compute the outputs at all positions in parallel. We still need to finish one block before starting the next, so L blocks give us L successive rounds of communication between positions.

The causal mask allows $T(T+1)/2$ query–key pairs per head. Computing the scores across all heads costs $O(HT^2d_{\text{head}}) = O(T^2d)$ operations. Multiplying the attention weights by the values

has the same cost. Adding the projections and feed-forward network gives a total cost of

$$O(L(Td^2 + T^2d)) \quad (15)$$

for the blocks when $d_{\text{ff}} = \Theta(d)$. Computing vocabulary logits at every position adds $O(T|\mathcal{V}|d)$. Notice that increasing the context length does not add parameters to the blocks, but it does increase computation. Comparing the T^2d attention term with the Td^2 projection and feed-forward term gives a ratio of T/d . Which cost dominates therefore depends on how long the sequence is relative to the model width.

A direct implementation would store a $T \times T$ attention matrix for each head. We can avoid storing the full matrix by computing attention in smaller blocks. FlashAttention [8] uses this idea to reduce memory traffic while computing the same dense attention operation. It saves memory, but the arithmetic cost in (15) still contains the quadratic term.

Decoding. How does the computation change when we generate text? For a prompt $x_{1:T}$, we first process all T positions in parallel and use h_T to predict x_{T+1} . This prompt pass is called *prefill*. After we choose x_{T+1} , we append it to the prefix and compute the distribution for x_{T+2} . We repeat until we produce an end-of-sequence token or reach a length limit. Each prediction depends on the token just chosen, so ordinary autoregressive decoding proceeds one token at a time.

Recomputing the entire prefix at every step would repeat work. By causality, appending a token does not change the representations at earlier positions. At each layer we can therefore retain the keys and values already computed in a *key-value cache*, or *KV cache* [10, Section 2.4]. For the new position, we compute a query, key, and value, append its key and value to that layer’s cache, and use the query to attend to the entire cache. The resulting vector continues through the block and on to the next layer. After the final layer, the readout supplies the next-token probabilities. The query changes at each new position, so we recompute its scores and softmax weights against the cached keys.

Once positions $1, \dots, t-1$ are cached, processing position t requires projections and a feed-forward computation for just one position, together with attention over t positions. Including the vocabulary readout, this costs

$$O(L(d^2 + td) + |\mathcal{V}|d).$$

Across all heads, the keys and values at one position have total width $2Hd_{\text{head}} = 2d$. Caching t positions in L layers therefore uses $2Ltd$ numbers per sequence. Thus caching trades memory for avoiding repeated projections and feed-forward computations; each new query still reads the growing prefix.

References

- [1] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin. *A Neural Probabilistic Language Model*. Journal of Machine Learning Research, 3:1137–1155, 2003. [Full paper](#).
- [2] D. Bahdanau, K. Cho, and Y. Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR, 2015; first circulated in 2014. [Full paper](#).
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. *Attention Is All You Need*. NeurIPS, 2017. [Full paper](#).
- [4] J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864, first circulated in 2021. [Full paper](#).

- [5] K. He, X. Zhang, S. Ren, and J. Sun. *Deep Residual Learning for Image Recognition*. CVPR, 2016. [Full paper](#).
- [6] J. L. Ba, J. R. Kiros, and G. E. Hinton. *Layer Normalization*. arXiv:1607.06450, 2016. [Full paper](#).
- [7] R. Xiong et al. *On Layer Normalization in the Transformer Architecture*. ICML, 2020. [Full paper](#).
- [8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022. [Full paper](#).
- [9] N. Elhage et al. *A Mathematical Framework for Transformer Circuits*. Transformer Circuits Thread, 2021. [Full article](#).
- [10] N. Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. arXiv:1911.02150, 2019. [Full paper](#).